

ML-SPR Algorithm Design for IQ-TREE 3

Detailed topology-search design for an IQ-TREE 3 fork — 21 July 2026

1. Recommended algorithm at a glance

Implement a **bounded-radius maximum-likelihood SPR hill climber**. Generate legal candidates from the current tree without mutating it; screen them cheaply; exactly re-evaluate a bounded shortlist using reversible topology changes; commit one best improving move; then repeat.

First-release policy: unrooted binary single-tree analyses, a configurable small radius, serial evaluation, one accepted move per round, and exact full-branch reoptimization after a move is committed.

```
current tree
→ enumerate legal bounded SPR moves
→ provisional local scoring of all moves
→ exact scoring of top K moves
→ commit one move only if it improves exact likelihood
→ reoptimize branches and update all search state
→ repeat until no improvement
```

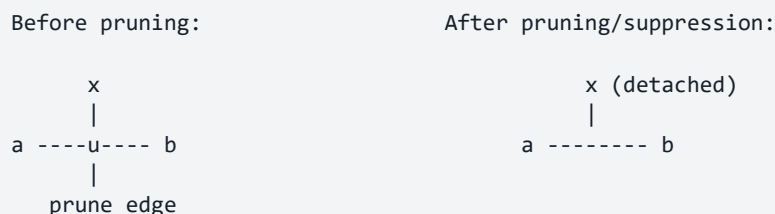
2. Tree and move model

2.1 Assumptions

- The first implementation supports connected, unrooted, binary trees.
- Each internal node has degree three; a tree with n leaves has $2n - 3$ edges.
- An SPR move is specified by a directed prune edge and an undirected target edge.
- Target edges inside the pruned component are illegal because they create a cycle or otherwise fail to change the intended topology.

2.2 One unrooted SPR move

Select directed edge $u \rightarrow x$, where x is the root of the subtree to detach and u is the former attachment node. The remaining two neighbors of u are a and b .



Suppressing u connects a and b . A safe initial length is:

$$L(a,b) = L(a,u) + L(u,b)$$

Next select a legal target edge $r-s$, split it by placing/reusing u , and attach the detached subtree through $u-x$.

Target before regraft:

r ----- s

Target after regraft:

```

r
 \
  u ---- x
 /
s
```

The target length can initially be divided equally, or by a more informed local optimizer:

$$L(r,u) = L(u,s) = L(r,s) / 2$$

All affected lengths are subsequently optimized. The initial split is an initialization, not an ML estimate.

3. Candidate generation

3.1 Why it needs a bound

There are roughly $2(2n - 3)$ directed prune choices and $O(n)$ target edges per choice. An unrestricted neighborhood is therefore $O(n^2)$ candidates per round before likelihood evaluation. Full likelihood calculations make that cost prohibitive for ordinary large-tree searches.

Define `spr_radius` as the number of tree edges between the former attachment region and the target edge in the remaining tree. A bounded radius yields a local neighborhood whose practical size is approximately $O(n \times 2^{\text{radius}})$ in a binary tree.

3.2 Generation procedure

1. Enumerate each possible directed prune edge $u \rightarrow x$.
2. Find the component rooted at x when the edge is cut, without modifying the tree.
3. Traverse the complement starting at the former u attachment region using BFS.
4. Collect target edges reached within the configured radius.
5. Exclude the original attachment edge, targets in the detached component, and moves that reproduce the original unrooted topology.
6. Canonicalize and deduplicate move descriptions.
7. Reject constraint-incompatible moves before likelihood scoring where possible.

```
generateCandidates(tree, radius):
    result = empty set
    for each directed edge (parent u, child x):
        pruned = markComponent(x, blocked=u)
        for each edge (r, s) in BFS_edges(start=u, max_distance=radius):
            if r or s is in pruned: continue
            move = canonicalize(u, x, r, s)
            if isIdentity(move) or move in result: continue
            if violatesConstraint(move): continue
            result.add(move)
    return result
```

3.3 Candidate identity and deduplication

The same final unrooted topology can be discovered by more than one traversal direction. Use a canonical representation based on the normalized prune directed edge plus a normalized target endpoint pair, or verify uniqueness with split hashes in debug/test mode. Regenerate candidates after every accepted move; candidate pointers/iterators should never survive a committed topology mutation.

4. Transactional topology operations

All likelihood evaluation depends on one core rule: temporary candidates must be reversible. Separate immutable move intent from mutable rollback state.

```
SPRMove
  prune_parent, prune_child
  target_left, target_right
  radius
  screening_score, exact_score

SPRRollback
  original endpoints and reciprocal neighbor state
  original lengths for all rewired edges
  original root/direction state if relevant
  branch-length snapshot if temporary optimization changes lengths
```

4.1 Required operations

Operation	Must do	Must not do
isLegalSPR	Check membership, binary assumptions, target outside pruned component, identity, and constraints.	Mutate the tree.
applySPR	Record every affected reciprocal link and length, suppress old attachment, split target, reconnect subtree.	Rely on later inference to recover old state.
rollbackSPR	Restore the recorded original state exactly.	Attempt to infer old adjacency from a changed tree.
scoreSPR	Apply, invalidate, optimize, calculate, and roll back.	Leak a temporary candidate into checkpoints or candidate trees.
commitSPR	Apply a chosen exact move, optimize, and update score/search state.	Commit from a screening score.

4.2 Structural invariants

- Every neighbor link is reciprocal.
- The tree remains connected and acyclic.
- Taxon IDs and names never change.
- No degree-two internal node remains after suppression/regrafting.
- Edge count remains $2n - 3$ for an unrooted binary tree.

- Applying and rolling back a candidate restores Newick topology, branch lengths, and likelihood within numeric tolerance.

5. Likelihood evaluation

5.1 Why SPR is harder than NNI

NNI changes a fixed local five-branch neighborhood. An SPR move changes local topology at both ends and can alter the partial-likelihood dependency path between them. An optimization that reuses stale partial likelihoods can silently choose an incorrect tree. Correctness must precede incremental-cache optimization.

5.2 Conservative baseline

1. Apply one temporary candidate.
2. Clear all partial likelihoods using the established tree routine.
3. Optimize branches according to the evaluation mode.
4. Compute likelihood through the normal exact tree likelihood path.
5. Roll back topology and branch lengths.
6. Clear/recompute caches as required so the restored tree cannot read partials from the temporary topology.

This may be expensive, but it creates a trustworthy reference implementation for later local invalidation.

5.3 Two-stage scoring

Stage	Purpose	Branch optimization	Use
Screening	Rank many candidates cheaply.	Attachment branch plus the branches created by suppressing/splitting edges.	Only to retain top K candidates.
Exact	Decide whether a candidate can be committed.	Initially optimize all branches; later use a proven equivalent affected-region procedure.	Acceptance decision only.

```

evaluateCandidate(move, mode):
    rollback = applySPR(move)
    clearAllPartialLH()
    if mode == SCREEN:
        optimizeLocalSPRBranches(move)
    else:
        optimizeAllBranches()
    score = computeLikelihood()
    rollbackSPR(rollback)
    clearAllPartialLH()
    return score

```

Acceptance rule: a candidate is accepted only when its exact score is greater than `curScore + loglh_epsilon`. Screening scores are never accepted directly.

6. Search loop and score ownership

```
curScore = computeExactLikelihood(current_tree)

for round in 1 .. max_rounds:
    candidates = generateCandidates(current_tree, spr_radius)
    screened = priorityQueue(capacity = top_k)

    for move in candidates:
        move.screening_score = evaluateCandidate(move, SCREEN)
        screened.insert_if_top_k(move)

    best = none
    for move in screened:
        move.exact_score = evaluateCandidate(move, EXACT)
        if best is none or move.exact_score > best.exact_score:
            best = move

    if best is none or best.exact_score <= curScore + epsilon:
        stop with "no improving bounded SPR move"

    commitSPR(best)
    optimizeAllBranches()
    curScore = computeExactLikelihood(current_tree)
    assert curScore > previous_score + epsilon
    update best tree, candidate trees, checkpoint, report statistics
```

Commit one move per round. Applying several apparently non-overlapping moves is unsafe initially because two SPRs can interact through pruned components, target paths, shared branches, and likelihood-cache dependencies.

7. Constraints, roots, and derived tree types

7.1 Constraint trees

The safe first approach is to apply a temporary candidate, invoke a tree-level constraint compatibility check, then roll back. This is slower than split-based prefiltering but is straightforward to validate. Once the core search is stable, precompute candidate split changes to reject incompatible moves before likelihood evaluation.

7.2 Rooted/nonreversible models

Directional partial likelihoods and root direction must be updated deliberately. If the base mutation API does not prove this correct, reject SPR for such analyses in version 1 rather than silently using the unrooted logic.

7.3 Supertrees, partitions, and mixtures

Base `PhyloTree` rewiring cannot automatically maintain linked partition trees, supertree neighbor mappings, or mixture-specific branch lengths. The first implementation should explicitly reject unsupported derived tree types. Later support requires subclass overrides comparable to the existing NNI overrides.

8. Complexity and tuning

Component	Unbounded behavior	Bounded version
Candidate count	$O(n^2)$ per round	Approximately $O(n \times 2^r)$ for radius r
Screening cost	One local likelihood evaluation per candidate	Limited by radius; further limited by optional sampling later
Exact cost	Potentially one full branch optimization per candidate	Limited to top K screened moves
Rounds	Can be large in a naive hill climber	Limited by no-improvement, round cap, and time budget

Recommended initial tuning is intentionally conservative: radius 3–5, exact shortlist 10–20, one committed move per round, and a finite round cap. Benchmark likelihood gain per added wall-clock time before enlarging defaults.

9. Test design

1. **Exhaustive small-tree move tests:** enumerate every legal bounded move; apply; validate; roll back; compare topology and lengths with a snapshot.
2. **Score-restoration tests:** original likelihood before temporary evaluation equals likelihood after rollback within tolerance.
3. **Monotonicity tests:** every committed move improves a fresh full likelihood calculation.
4. **Candidate tests:** no target inside pruned component; no duplicates; known counts on hand-enumerated trees.
5. **Constraint tests:** invalid candidates are never committed.
6. **Determinism tests:** fixed input/options/seed produce identical final topology and score.
7. **Regression tests:** SPR-disabled runs reproduce existing NNI-only behavior.
8. **Resume tests:** a checkpoint at a committed round boundary resumes to the same result as an uninterrupted run.

10. Repository implementation locations

Concern	Primary location	Action
Move structures and low-level APIs	<code>tree/phylo tree.h</code>	Replace/extend legacy <code>SPRMove</code> ; add rollback and mutation/evaluation declarations.
Topology mutation and likelihood evaluation	<code>tree/phylo tree.cpp</code>	Implement transactional apply/rollback and conservative score evaluation; use legacy routines only as reference.
Search orchestration	<code>tree/iq tree.h</code> , <code>tree/iq tree.cpp</code>	Add SPR search loop alongside mature NNI search lifecycle.
Workflow invocation	<code>main/phylo analysis.cpp</code>	Call integrated SPR refinement/search instead of direct prototype code.
CLI configuration	<code>utils/tools.h</code> , <code>utils/tools.cpp</code>	Expose radius, top K, rounds, epsilon, and mode; remove hard-coded behavior.
Reference implementation	<code>pll/searchAlgo.c</code>	Study its candidate ranking and rollback patterns; do not merge its tree representation directly.

11. Optimization sequence after correctness

1. Full cache clearing and exact all-branch evaluation.
2. Bounded candidate radius and top-K exact shortlist.
3. Local branch optimization for screening.
4. Proven local partial-likelihood invalidation along old/new attachment paths.
5. Candidate prescreening with parsimony, distance, or a safe likelihood bound.
6. Parallel exact evaluation only with independent temporary trees or rigorously isolated state.

The governing principle is simple: a slower candidate evaluator that is exactly reversible and scores the right topology is a useful baseline; a faster evaluator that leaves stale partial likelihoods is not.